QuarkyLab GPU Cluster

Student User Guide

A complete, step-by-step guide to connecting to the cluster
and running your first GPU jobs

No prior experience with clusters, Linux, or job schedulers is assumed. If you can open a terminal and follow steps in order, you can use QuarkyLab.

NetFRAME Home Lab

Version 1.0 • July 2, 2026

Contents

1 Introduction

QuarkyLab is a shared computer built for machine-learning and scientific computing. At its heart is a powerful graphics processing unit (a NVIDIA RTX 8000 (48 GB) *GPU*) that many students use at the same time. Because everyone shares one GPU, you do not run your programs on it directly. Instead, you *submit* your work to a program called a **scheduler**, which lines up everyone's jobs and runs them fairly, one slice at a time.

This guide takes you from a blank computer to running your first GPU program, in order, with nothing skipped. Here is the whole journey at a glance:

1. **Create an identity key** on your own computer (a one-time setup).
2. **Join the private network** that QuarkyLab lives on (a secure VPN).
3. **Connect** to QuarkyLab with a single command.
4. **Learn where your files live** on the cluster.
5. **Submit your first GPU job** and read its results.
6. **Manage your jobs** and work within the shared limits.

Note

Throughout this guide, text you must replace with your own value is shown in angle brackets, e.g. `<your-username>`. Your administrator assigns you a username of the form `studentNN` (for example `student07`). Wherever you see `studentNN`, use the one you were given.

1.1 What you will need

- A laptop or desktop running **Windows**, **macOS**, or **Linux**.
- A working internet connection.
- A username on QuarkyLab (your administrator provides this).
- About 20 minutes for the one-time setup.

1.2 A word on the terminal

Almost everything in this guide happens in a **terminal** (also called a *command line* or *shell*) — a window where you type commands instead of clicking buttons. Opening one differs slightly per platform:

- **Windows:** press the Start key, type `PowerShell`, and press Enter.
- **macOS:** press `Cmd+Space`, type `Terminal`, and press Enter.
- **Linux:** press `Ctrl+Alt+T`, or search for `Terminal` in your applications.

When this guide shows a command in a shaded box, type it exactly (or copy–paste it) and press Enter.

2 Step 1 — Create your SSH key

To prove who you are when connecting, QuarkyLab uses an **SSH key** instead of a password. A key comes in two halves:

- a **private key** (a secret file that stays on your computer — never share it), and

- a **public key** (a short line of text you *do* share, so the cluster recognises you).

You will generate both now, then send only the *public* half to your administrator.

Windows, macOS, and Linux (same command)

Open your terminal and run:

```
ssh-keygen -t ed25519 -C "<your-name>@quarkylab"
```

You will be asked three things:

1. *"Enter file in which to save the key"* — press **Enter** to accept the default location.
2. *"Enter passphrase"* — type a passphrase (a password protecting the key). **Recommended.** You will type it when you connect.
3. *"Enter same passphrase again"* — type it once more.

This creates two files in a hidden `.ssh` folder in your home directory: `id_ed25519` (private — keep secret) and `id_ed25519.pub` (public — to share).

2.1 Display and send your public key

Show the public key so you can copy it:

macOS / Linux

```
cat ~/.ssh/id_ed25519.pub
```

Windows (PowerShell)

```
type $env:USERPROFILE\.ssh\id_ed25519.pub
```

The output is a single line beginning with `ssh-ed25519`. Copy that entire line and email it to your administrator at **masonkr@gmail.com**, telling them your name and (if known) your assigned `studentNN` username.

Important

Send only the `.pub` (public) line. **Never send or share the private key** (`id_ed25519`, no `.pub`). Anyone with your private key can log in as you.

Note

Your administrator adds your key to the cluster. Until they confirm it is added, the connection in Step 3 will be refused — this is normal. Wait for their confirmation before continuing past Step 2.

3 Step 2 — Join the private network (VPN)

QuarkyLab is not exposed to the open internet. To reach it from anywhere, you join a private, encrypted network using a free app called **Tailscale**, connected to our own coordination server (**Headscale**). This is a one-time install plus a login.

3.1 Install the Tailscale app

Windows

Download and run the installer from https://tailscale.com/download/windows. Accept the defaults. A Tailscale icon appears in your system tray.

macOS

Install from the Mac App Store (search *Tailscale*) or https://tailscale.com/download/mac. A Tailscale icon appears in your menu bar.

Linux

Run the official installer, then start the service:

```
curl -fsSL https://tailscale.com/install.sh | sh
```

3.2 Connect to our network

Instead of Tailscale's default sign-in, point the app at our coordination server. Your administrator will give you either a **sign-up link** or a one-time **authentication key**.

Linux (and macOS/Windows with the CLI installed)

```
sudo tailscale up --login-server https://headscale.kylemason.org
```

This prints a URL. Open it in your browser and follow the prompts to register your device. If your administrator gave you an authentication key instead, use:

```
sudo tailscale up --login-server https://headscale.kylemason.org --authkey <key-from-
    admin>
```

Windows / macOS (graphical app)

Right-click the Tailscale icon and choose *Preferences* (or *Use custom coordination server* / *Change server*). Enter:

```
https://headscale.kylemason.org
```

Then click *Log in* and complete the prompts your administrator described.

3.3 Confirm you are connected

```
tailscale status
```

You should see QuarkyLab (or `quarkylab`) listed. As a final check, make sure you can reach it:

```
ping quarkylab
```

Replies mean the network is up. (Press `Ctrl+C` to stop `ping`.)

Tip

Leave Tailscale running whenever you want to use QuarkyLab. If a connection later "times out," the first thing to check is that Tailscale is connected (`tailscale status`).

4 Step 3 — Connect to QuarkyLab

With your key added (Step 1) and the VPN up (Step 2), connect with one command. Use *your* assigned username in place of `studentNN`:

```
ssh studentNN@quarkylab
```

- The **first time only**, you will see *"The authenticity of host … can't be established … (yes/no)?"* Type `yes` and press Enter. This remembers the server so you are not asked again.
- If you set a passphrase in Step 1, enter it now. (It is your *key* passphrase, not a cluster password.)

When you succeed, you land on the **login node** and see a short welcome banner. You are now "on" QuarkyLab.

Important

The login node is a shared front desk, not a workbench. **Do not run heavy training or GPU code directly here.** All real work is submitted as a *job* (Step 5). Interactive sessions (`srun`/`salloc`) are disabled for students.

4.1 If something goes wrong

Symptom	Most likely fix
`Permission denied (publickey)`	Your key is not added yet, or you used the wrong username. Confirm with your administrator; check you typed `studentNN` correctly.
`Connection timed out`	The VPN is not connected. Run `tailscale status`; reconnect (Step 2).
`Could not resolve hostname`	Same as above — the VPN provides the name `quarkylab`.
Asked for a *password*	Password login is off. You must connect with your SSH key; re-check Step 1.

5 Step 4 — Where your files live

QuarkyLab gives you three distinct storage areas. Knowing what each is for will save you a lot of trouble.

Location	Purpose	Key facts
~ (your home)	Your code and results	100 GB quota. **Backed up nightly.** Keep anything you want to keep here.
`/scratch`	Fast temporary space (inside jobs)	**Not backed up**, auto-deleted after 14 days. Use for checkpoints and intermediate files.
`/data/shared`	Shared datasets & docs	**Read-only.** You cannot modify it.

Tip

Put large temporary files and training checkpoints in `/scratch` — it is fast and does not count against your home quota. Put final results and code in your home so they are backed up.

5.1 Copying files to and from the cluster

Run these from *your own* computer's terminal (not from inside the cluster). Replace `studentNN` with your username.

Upload a file to your home directory

```
scp mydata.csv studentNN@quarkylab:~/
```

Download a results file back to your computer

```
scp studentNN@quarkylab:~/myjob-1234.out .
```

Copy an entire folder (upload)

```
rsync -av myproject/ studentNN@quarkylab:~/myproject/
```

Note

Prefer a graphical editor? **Visual Studio Code** with the *Remote — SSH* extension lets you edit files on QuarkyLab as if they were local, using the same `studentNN@quarkylab` address. It is optional but popular.

6 Step 5 — Run your first GPU job

You do not run GPU code by hand. You write a short **job script**, hand it to the scheduler with `sbatch`, and the scheduler runs it for you — inside a ready-made container that already has PyTorch, TensorFlow, and a GPU slice attached. Output is written to a file you can read afterwards.

6.1 Write a tiny program

On the login node, create `train.py` (use the `nano` editor: type `nano train.py`, paste, then `Ctrl+O`, Enter, `Ctrl+X`):

```
import torch
print("GPU:", torch.cuda.get_device_name(0))
print("CUDA available:", torch.cuda.is_available())
# ... your real training code goes here ...
```

6.2 Write the job script

Create `train.sh` next to it. The lines starting with `#SBATCH` are instructions to the scheduler.

```
#!/bin/bash
#SBATCH -J myjob            # a name for your job
#SBATCH -t 01:00:00         # time limit HH:MM:SS (max 2 hours)
#SBATCH -o myjob-%j.out     # output file (%j becomes the job ID)

source /opt/conda/etc/profile.d/conda.sh
conda activate ml
python train.py
```

6.3 Submit it

```
sbatch train.sh
```

The scheduler replies with a job ID, e.g. `Submitted batch job 1234`. Your job is now queued; it starts as soon as a GPU slice is free.

6.4 Watch it and read the result

```
squeue --me                 # is my job running or waiting?
cat myjob-1234.out          # read the output once it finishes
```

Note

Everything happens automatically inside a private, isolated container: the `ml` environment (Python 3.11, PyTorch, TensorFlow, and more), a GPU slice with 6 GB of memory, CPU cores, and RAM. You only write the two files above.

7 Managing your jobs

Command	What it does
`squeue --me`	List your running and queued jobs
`sacct`	Show your recent job history (and whether they succeeded)
`scancel <jobid>`	Cancel a job you submitted
`scontrol show job <jobid>`	Show detailed status for one job
`cat myjob-<jobid>.out`	View a finished job's output

8 Limits and rules

QuarkyLab is shared, so a few limits keep it fair for everyone.

Limit	Value
GPU memory per job (hard cap)	6 GB
Running jobs at once	1 (plus 3 more queued)
Maximum job time	2 hours
Internet access inside a job	none
Interactive `srun`/`salloc`	disabled — use `sbatch`
Whole-GPU access	researchers only

8.1 Need more GPU memory?

GPU memory is handed out in *shards* of 6 GB each. Ask for more by adding one line to your job script (two shards = 12 GB):

```
#SBATCH --gres=shard:2      # request 12 GB of GPU memory
```

You cannot request the entire card — that is reserved for researchers.

9 Common problems and fixes

- **"No internet in my job" / `pip install` fails.** Jobs have no network by design. The `ml` environment already includes the common libraries. Need another package? Ask your administrator to add it to the image.
- **`CUDA out of memory`.** You have 6 GB. Reduce your batch size or model size, or request more shards (above).
- **My job disappeared / was requeued.** If a researcher needs the whole GPU, student jobs are paused and restarted automatically. **Save checkpoints** to `/scratch` so you can resume.
- **My job hit the time limit.** Jobs stop at 2 hours. Checkpoint long runs and resubmit to continue.
- **`Permission denied` when connecting.** Your key is not added yet, or the username is wrong (see Step 3).

10 Tools available inside the lab

When your job runs, it lands in a container preloaded with a scientific Python stack (the `ml` environment) plus standard cluster tools. The essentials:

Machine-learning & scientific Python (`ml` environment)

- **Python 3.11**
- **PyTorch 2.5** (CUDA-enabled) — deep learning
- **TensorFlow 2.21** — deep learning
- **NumPy, SciPy, pandas** — arrays, math, data frames
- **scikit-learn** — classical machine learning
- **Hugging Face `transformers` & `datasets`** — pretrained models and data

Cluster & command-line tools

- **SLURM** commands: `sbatch`, `squeue`, `sacct`, `scancel`, `scontrol`
- **conda** — to activate the `ml` environment
- File tools: `scp`, `rsync`, `nano`, plus the usual `ls`, `cd`, `cat`, `mkdir`
- **NVIDIA CUDA** runtime — so PyTorch/TensorFlow can use the GPU

Note

Need a package that is not listed? It cannot be installed at job time (no internet in jobs). Email masonkr@gmail.com and ask for it to be added to the container image.

11 Understanding SLURM (and why it matters for your career)

The scheduler you have been using is called **SLURM** (Simple Linux Utility for Resource Management). It is worth understanding what it is, because it is a genuinely transferable, career-relevant skill.

What SLURM is

SLURM is a **job scheduler** and **resource manager**. When many people share a small number of expensive machines (like our single NVIDIA RTX 8000 (48 GB)), something has to decide *who runs what, when, and with how much of the hardware*. SLURM is that "something." You describe your job once — what to run, how long, how much memory — and SLURM finds a fair, efficient time to run it, then reports back. This is why you submit with `sbatch` rather than running directly: it lets twenty students share one GPU without stepping on each other.

Why it exists here

Our GPU is a shared, finite resource. Without a scheduler, whoever started first would monopolise it and everyone else would be blocked. SLURM enforces fair sharing (each student gets turns), hard limits (so one job cannot consume the whole machine), and priorities (researchers' work can take precedence when needed). The result is that a single GPU serves a whole class reliably.

Where you will meet SLURM again

SLURM is the *de facto* standard for batch computing in research and industry. Learning it here means you are already fluent in a tool used across:

- **University research computing** — nearly every campus HPC cluster (and national programs such as ACCESS) runs SLURM. Graduate research in almost any computational field will expect it.
- **National laboratories & supercomputers** — systems like NERSC's Perlmutter and the DOE leadership machines schedule work with SLURM. If you ever run large simulations or model training at scale, this is how.
- **Industry ML & AI** — companies training large models operate GPU clusters managed by SLURM (or close relatives) to share thousands of GPUs among teams.
- **Scientific & engineering computing** — bioinformatics pipelines, computational chemistry, climate and weather modelling, computational fluid dynamics, and finance risk analysis all commonly run on SLURM-managed clusters.

- **Cloud HPC** — managed offerings (e.g. AWS ParallelCluster, Azure CycleCloud) provision SLURM clusters on demand, so the same `sbatch` workflow follows you into the cloud.

Even where a different scheduler is used (PBS/Torque, LSF, or Grid Engine), the concepts are the same — submit a job script, request resources, check the queue, read the output. The habits you build on QuarkyLab transfer directly.

12 Getting help

Stuck, or need a package added to the environment? Contact the cluster administrator at **masonkr@gmail.com**. When you write, include your `studentNN` username, the command you ran, and any error message you saw — it makes help much faster.

Welcome to QuarkyLab. Happy computing.

A Quick command reference

Command	Purpose
`ssh-keygen -t ed25519`	Create your SSH key (one time)
`tailscale up --login-server https://headscale.kylemason.org`	Join the private network
`ssh studentNN@quarkylab`	Log in to the cluster
`scp <file> studentNN@quarkylab:~/`	Upload a file
`sbatch train.sh`	Submit a job
`squeue --me`	See your jobs
`sacct`	See job history
`scancel <jobid>`	Cancel a job

B Glossary

SSH Secure Shell — the secure way you connect to and control a remote computer from a terminal.

SSH key A pair of files (private + public) that proves your identity in place of a password.

VPN Virtual Private Network — a secure, private tunnel that lets your computer reach QuarkyLab from anywhere.

Node A single computer in the cluster. You first land on the *login node*.

Scheduler The program (SLURM) that queues and runs everyone's jobs fairly.

Job A unit of work you submit with `sbatch`, described by a small job script.

Partition A named queue of jobs (you use the `student` partition).

GPU / shard The graphics processor used for training; a *shard* is a 6 GB slice of it.

Container A sealed, preloaded software environment your job runs inside, with all libraries ready.